

glfwdojo 開発ガイド

このドキュメントは、既存の機能を変更したり新しい要素を追加するときの手順と注意点をまとめたものです。

ビルド手順・ファイル追加時の CMakeLists.txt 設定は [BUILD.md](#) を参照してください。

目次

1. [アーキテクチャ概要](#)
2. [UBO \(Uniform Buffer Object \)](#)
3. [ライトの変更](#)
4. [ライトキューブの追加](#)
5. [新しい3Dオブジェクトの追加](#)
6. [インスタンスングの追加](#)
7. [新しいシェーダーの追加](#)
8. [テクスチャの追加](#)
9. [よくある落とし穴](#)

アーキテクチャ概要

```
main.cpp
├── Window      : GLFWウィンドウ管理・メインループ
├── Scene       : すべての描画処理を管理する中心クラス
│   ├── Shader : GLSLシェーダープログラムのラッパー
│   ├── Camera : 視点・ビュー行列の管理
│   ├── TextureCache : テクスチャの読み込みとキャッシュ (weak_ptr管理)
│   ├── Lighting.h : PointLight / DirectionalLight / SpotLight 構造体
│   └── GeometryData.h : 頂点データ・インデックス・位置配列の定義
```

描画の流れ (Render() 内)

```
カスタムフレームバッファへバインド
↓
UBO に view / projection を書き込む
↓
不透明オブジェクト ( cube, floor ) を描画
↓
スカイボックスを描画 ( GL_LEQUAL で最遠描画 )
↓
透明オブジェクトをカメラ距離でソートして描画
↓
デフォルトフレームバッファへ戻す
```

↓
スクリーンクワッドにカラーテクスチャを貼って描画

UBO (Uniform Buffer Object)

view と projection は UBO (binding = 0) で全シェーダーに共有されています。

C++ 側 (Scene.cpp の Render()) :

```
glBindBuffer(GL_UNIFORM_BUFFER, ubo_);
glBufferSubData(GL_UNIFORM_BUFFER, 0, sizeof(glm::mat4),
glm::value_ptr(view));
glBufferSubData(GL_UNIFORM_BUFFER, sizeof(glm::mat4), sizeof(glm::mat4),
glm::value_ptr(projection));
glBindBuffer(GL_UNIFORM_BUFFER, 0);
```

GLSL 側 (各 .vert ファイル) :

```
layout (std140, binding = 0) uniform Matrices {
    mat4 view;          // offset 0
    mat4 projection;    // offset 64
};
```

注意: メンバの順序 (view → projection) と C++ の書き込み順を必ず一致させること。新しい .vert ファイルを作るときも同じブロック定義をコピーする。

ライトの変更

PointLight の位置を変更する

GeometryData.h の pointLights 配列を直接編集します :

```
std::array<gl::PointLight, 4> pointLights = {{
    { glm::vec3(1.0f, 2.0f, 0.0f) }, // ← ここを変更
    { glm::vec3(2.3f, -3.3f, -4.0f) },
    { glm::vec3(-4.0f, 2.0f, -12.0f) },
    { glm::vec3(0.0f, 0.0f, -3.0f) },
}};
```

各 PointLight のメンバ (Lighting.h 参照) :

```
struct PointLight {
    glm::vec3 position;
```

```
glm::vec3 ambient = { 0.05f, 0.05f, 0.05f };
glm::vec3 diffuse = { 1.0f, 1.0f, 1.0f };
glm::vec3 specular = { 1.0f, 1.0f, 1.0f };
float constant = 1.0f;
float linear = 0.09f;
float quadratic = 0.032f;
};
```

ライトをシェーダーに送る（コメントアウト中の復元）

`Render()` 内でコメントアウトされているライト送信ブロックを有効にします：

```
// DirectionalLight
directionalLight_.applyToShader(*shader_, "dirLight");

// PointLight (4個ループ)
for (size_t i = 0; i < pointLights.size(); ++i) {
    pointLights[i].applyToShader(*shader_, "pointLights[" +
std::to_string(i) + "]");
}

// SpotLight (カメラ追従)
spotlight_.applyToShader(*shader_, "spotLight", *camera_);
```

前提: `shader_` を `use()` した後に呼ぶこと。

ライトキューブの追加

ライトの位置に小さなキューブを描画してライト位置を可視化する手順です。

1. Scene.h にメンバを追加

```
unsigned int lightVAO_;
unsigned int lightVBO_; // cubeVBO_ と共有してもよい
std::unique_ptr<gl::Shader> lightShader_;
```

2. コンストラクタでシェーダーを生成

```
lightShader_ = std::make_unique<gl::Shader>("light_cube.vert",
"light_cube.frag");
```

3. initMesh() でVAOを設定

キューブと同じ頂点データを使い、`position (location=0)` だけ有効化します：

```

glGenVertexArrays(1, &lightVAO_);
glBindVertexArray(lightVAO_);
glBindBuffer(GL_ARRAY_BUFFER, cubeVBO_); // cubeVBO_ を再利用
glEnableVertexAttribArray(0);
glVertexAttribPointer(0, 3, GL_FLOAT, GL_FALSE, sizeof(glm::Vertex),
    (void*)offsetof(glm::Vertex, position));
glBindVertexArray(0);

```

4. Render() で描画 (不透明オブジェクト描画の末尾に追加)

```

lightShader_>use();
glBindVertexArray(lightVAO_);
for (const auto& light : pointLights) {
    glm::mat4 lightModel = glm::translate(glm::mat4(1.0f), light.position);
    lightModel = glm::scale(lightModel, glm::vec3(0.2f));
    lightShader_>setMat4("model", lightModel);
    glDrawElements(GL_TRIANGLES, gl::cubeIndices.size(), GL_UNSIGNED_INT,
0);
}

```

`glDrawElements` を使うには EBO が必要です。 `lightVAO_` に `cubeEBO_` を紐付けるか、
`glDrawArrays(GL_TRIANGLES, 0, 36)` を使う場合は重複頂点版の頂点データが必要です。

5. light_cube.vert の例

```

#version 460 core
layout (location = 0) in vec3 aPos;

layout (std140, binding = 0) uniform Matrices {
    mat4 view;
    mat4 projection;
};
uniform mat4 model;

void main() {
    gl_Position = projection * view * model * vec4(aPos, 1.0);
}

```

6. デストラクタに削除を追加

```

glDeleteVertexArrays(1, &lightVAO_);
// lightVBO_ を独立させた場合のみ:
// glDeleteBuffers(1, &lightVBO_);

```

新しい3Dオブジェクトの追加

1. GeometryData.h に頂点データを追加

```
inline const std::array<Vertex, 4> rawMyObjectVertices = { {  
    {{ x, y, z }, {}, { u, v }},  
    // ...  
} };  
inline const std::array<unsigned int, 6> myObjectIndices = { 0, 1, 2, 2, 3,  
0 };  
inline const std::array<Vertex, 4> myObjectVertices =  
calculateFaceNormals(rawMyObjectVertices);
```

法線は `calculateFaceNormals()` が自動計算します（4頂点/面の構造が前提）。

2. Scene.h にVAO/VBO/EBOを追加

```
unsigned int myObjectVAO_, myObjectVBO_, myObjectEBO_;
```

3. initMesh() でバインド設定

```
int stride = sizeof(gl::Vertex);  
glGenVertexArrays(1, &myObjectVAO_);  
glGenBuffers(1, &myObjectVBO_);  
glGenBuffers(1, &myObjectEBO_);  
glBindVertexArray(myObjectVAO_);  
glBindBuffer(GL_ARRAY_BUFFER, myObjectVBO_);  
glBufferData(GL_ARRAY_BUFFER, sizeof(gl::myObjectVertices),  
gl::myObjectVertices.data(), GL_STATIC_DRAW);  
glBindBuffer(GL_ELEMENT_ARRAY_BUFFER, myObjectEBO_);  
glBufferData(GL_ELEMENT_ARRAY_BUFFER, sizeof(gl::myObjectIndices),  
gl::myObjectIndices.data(), GL_STATIC_DRAW);  
glEnableVertexAttribArray(0);  
glVertexAttribPointer(0, 3, GL_FLOAT, GL_FALSE, stride,  
(void*)offsetof(gl::Vertex, position));  
glEnableVertexAttribArray(1);  
glVertexAttribPointer(1, 3, GL_FLOAT, GL_FALSE, stride,  
(void*)offsetof(gl::Vertex, normal));  
glEnableVertexAttribArray(2);  
glVertexAttribPointer(2, 2, GL_FLOAT, GL_FALSE, stride,  
(void*)offsetof(gl::Vertex, uv));  
glBindVertexArray(0);
```

4. Render() で描画

```
shader_>use();
glBindVertexArray(myObjectVAO_);
myTexture_>bind(0);
shader_>setMat4("model", glm::mat4(1.0f));
glDrawElements(GL_TRIANGLES, gl::myObjectIndices.size(), GL_UNSIGNED_INT,
0);
```

5. デストラクタに削除を追加

```
glDeleteVertexArrays(1, &myObjectVAO_);
glDeleteBuffers(1, &myObjectVBO_);
glDeleteBuffers(1, &myObjectEBO_);
```

インスタンスングの追加

同じメッシュを複数の位置に描画するときに使います。

1. GeometryData.h に位置配列を追加

```
inline const std::vector<glm::vec3> myObject_pos = {
    glm::vec3(0.0f, 0.0f, 0.0f),
    glm::vec3(3.0f, 0.0f, 0.0f),
};
```

2. Scene.h にインスタンスVBOを追加

```
unsigned int myObjectInstanceVBO_;
```

3. initMesh() でインスタンスVBOを設定 (VAO バインド中に行うこと)

```
glBindBuffer(GL_ARRAY_BUFFER, myObjectInstanceVBO_);
glBufferData(GL_ARRAY_BUFFER, sizeof(gl::myObject_pos),
gl::myObject_pos.data(), GL_STATIC_DRAW);
glEnableVertexAttribArray(3); // location = 3
glVertexAttribPointer(3, 3, GL_FLOAT, GL_FALSE, sizeof(glm::vec3),
(void*)0);
glVertexAttribDivisor(3, 1); // インスタンスごとに1回進む
glBindBuffer(GL_ARRAY_BUFFER, 0);
```

注意: `glVertexAttribDivisor` は必ず VAO がバインドされている間 (`glBindVertexArray(0)` の前) に呼ぶこと。

4. .vert シェーダーに instance 属性を追加

```
layout (location = 3) in vec3 aOffset;

void main() {
    mat4 instanceModel = mat4(1.0);
    instanceModel[3] = vec4(aOffset, 1.0); // 平行移動列を設定
    gl_Position = projection * view * instanceModel * vec4(aPos, 1.0);
}
```

5. Render() で描画

```
glDrawElementsInstanced(GL_TRIANGLES, gl::myObjectIndices.size(),
    GL_UNSIGNED_INT, 0, gl::myObject_pos.size());
```

毎フレーム位置が変わる場合 (glBufferSubData パターン)

`GL_DYNAMIC_DRAW` で確保し、毎フレーム `glBufferSubData` で更新します：

```
// initMesh(): GL_STATIC_DRAW → GL_DYNAMIC_DRAW に変更
glBufferData(GL_ARRAY_BUFFER, N * sizeof(glm::vec3), nullptr,
    GL_DYNAMIC_DRAW);

// Render(): ソート済み配列を毎フレーム書き込む
glBindBuffer(GL_ARRAY_BUFFER, myObjectInstanceVBO_);
glBufferSubData(GL_ARRAY_BUFFER, 0, sortedPositions.size() *
    sizeof(glm::vec3), sortedPositions.data());
glBindBuffer(GL_ARRAY_BUFFER, 0);
```

新しいシェーダーの追加

1. shader_src/ にファイルを作成

`myshader.vert` / `myshader.frag` を `shader_src/` に作成します。最低限のテンプレート (UBO 対応版)：

```
// myshader.vert
#version 460 core
layout (location = 0) in vec3 aPos;
layout (location = 2) in vec2 aTexCoords;
```

```

layout (std140, binding = 0) uniform Matrices {
    mat4 view;
    mat4 projection;
};
uniform mat4 model;

out vec2 TexCoords;

void main() {
    gl_Position = projection * view * model * vec4(aPos, 1.0);
    TexCoords = aTexCoords;
}

```

2. CMakeLists.txt の SHADER_SOURCES に追記

詳細は [BUILD.md](#) の「[新しいシェーダーファイルの追加](#)」を参照してください。

3. Scene.h にメンバを追加

```
std::unique_ptr<gl::Shader> myShader_;
```

4. コンストラクタで生成

```
myShader_ = std::make_unique<gl::Shader>("myshader.vert", "myshader.frag");
```

テクスチャの追加

通常テクスチャ

```

// Scene.h
std::shared_ptr<Texture> myTexture_;

// initTextures()
myTexture_ = cache_.get("resources\\textures\\filename.png", true);
// 第2引数: true = 上下反転あり (通常はtrue)、アルファ付きPNGも自動判別

// Render()
myTexture_->bind(0); // テクスチャユニット0にバインド

```

キューブマップ (スカイボックス用)

```

// 6面の画像パスを順番通りに渡す
std::vector<std::string> faces = {

```



```

    "resources\\textures\\skybox\\right.jpg",
    "resources\\textures\\skybox\\left.jpg",
    "resources\\textures\\skybox\\top.jpg",
    "resources\\textures\\skybox\\bottom.jpg",
    "resources\\textures\\skybox\\front.jpg",
    "resources\\textures\\skybox\\back.jpg",
};
cubemapTexture_ = cache_.loadCubemap(faces, false); // false = 反転なし

```

よくある落とし穴

`std::vector` に `sizeof` を使ってはいけない

```

// NG: vector オブジェクトのサイズ (24 bytes) が返る
glBufferData(GL_ARRAY_BUFFER, sizeof(gl::windows_pos), ...);

// OK: データの実際のバイト数
glBufferData(GL_ARRAY_BUFFER, gl::windows_pos.size() * sizeof(glm::vec3),
...);

```

`std::array` は `sizeof` でデータサイズが取れます。`std::vector` は必ず `.size() * sizeof(要素型)` を使うこと。

VAO の外で `glVertexAttribPointer` を呼ばない

VAO はバインド中に呼ばれた `glVertexAttribPointer` と `glVertexAttribDivisor` の設定を記録します。`glBindVertexArray(0)` の後に呼んでも VAO に記録されません。

ヘッダで変数定義するときは `inline` をつける

`GeometryData.h` のように複数の `.cpp` からインクルードされるヘッダで変数を定義するときは `inline` が必須です。ないと ODR (One Definition Rule) 違反でリンクエラーになります：

```

inline const std::vector<glm::vec3> myPositions = { ... }; // OK
const std::vector<glm::vec3> myPositions = { ... }; // NG ( 複数回定義エラー )

```

UBO のメンバ順序を統一する

複数のシェーダーが同じ UBO (binding=0) を使う場合、すべてのシェーダーで **メンバの順序・型が完全一致** している必要があります。順序が違っていると行列の値が入れ違いになります。

透明オブジェクトは必ず後から描画する

アルファブレンディングはフレームバッファに既にある色と合成します。透明オブジェクトを不透明オブジェクト・スカイボックスより先に描くと、背景が合成されません。

描画順: 不透明 → スカイボックス → 透明（後方から前方の順）

`unique_ptr` を初期化せずに使うとクラッシュ

コンストラクタでコメントアウトしたシェーダー生成を残したまま `Render()` で `->use()` を呼ぶと、`nullptr` 参照でメモリ違反になります。使わないシェーダーは `Render()` 側の呼び出しもコメントアウトするか、メンバ宣言ごと削除してください。